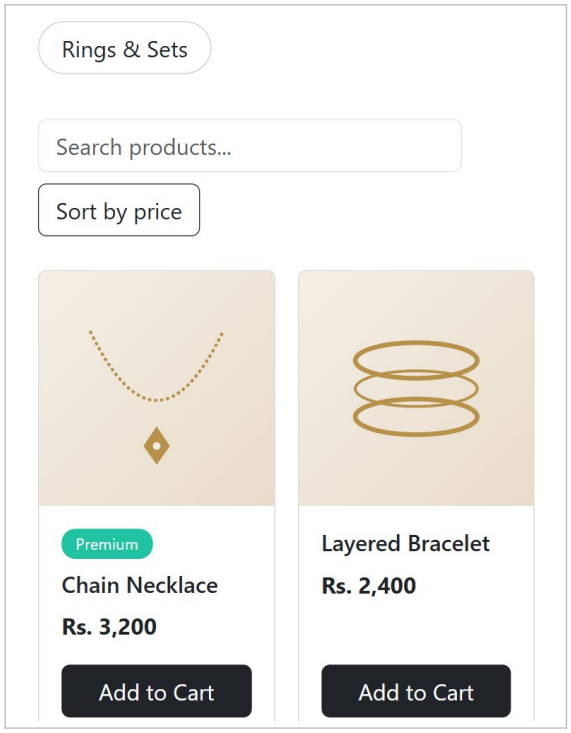
Abuzar Jamil

Registration No: FA24-BCS-124

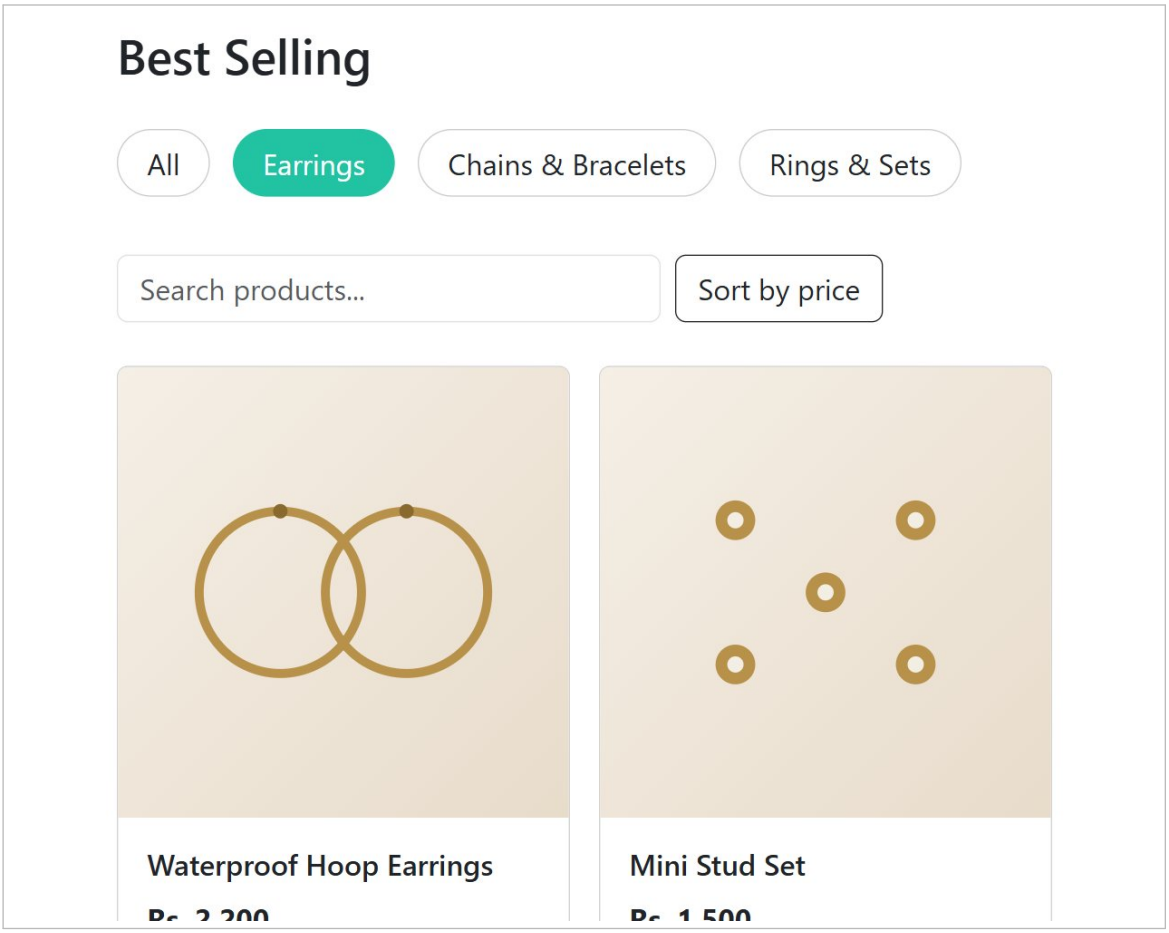
Web Technologies (CSC336) - Lab Mid Term - Upgrade Your E-commerce Listing and Product Pages
Personal numbers: N = 24, BP = 624px, H = 168 (accent hsl(168, 70%, 45%)), K = 3, sticky offset = 94px, Premium limit = Rs. 2,400

Part 1 - Screenshots (375px, 624px, 1280px)

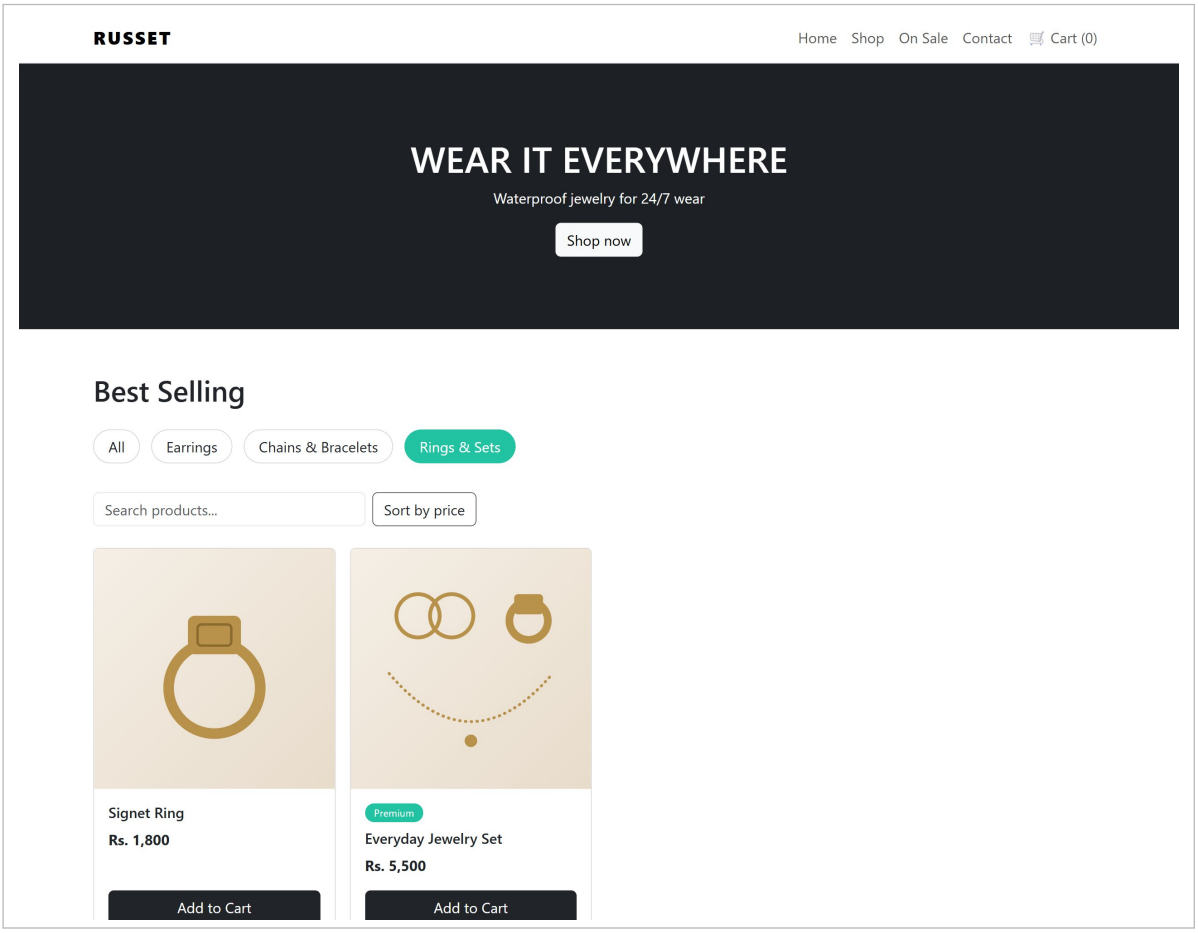
Task 1 - CSS-only category filter



Task 1 - 375px: Chains & Bracelets selected, selected label in accent colour

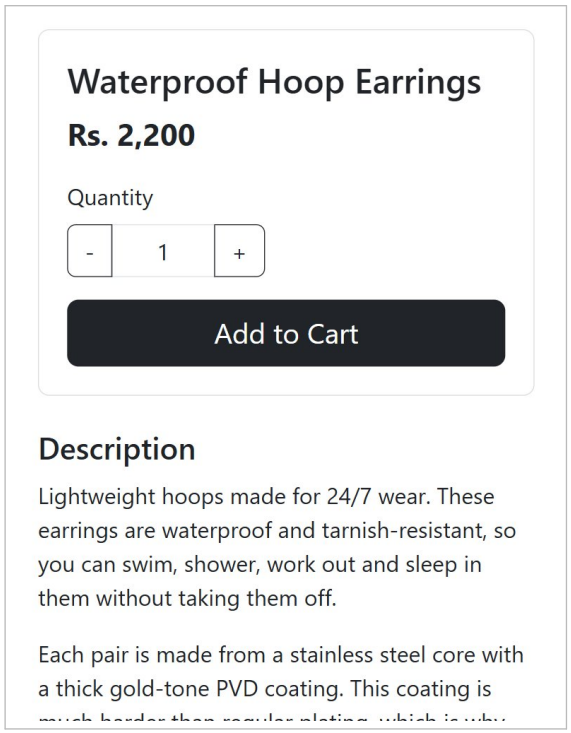


Task 1 - 624px (BP): Earrings selected

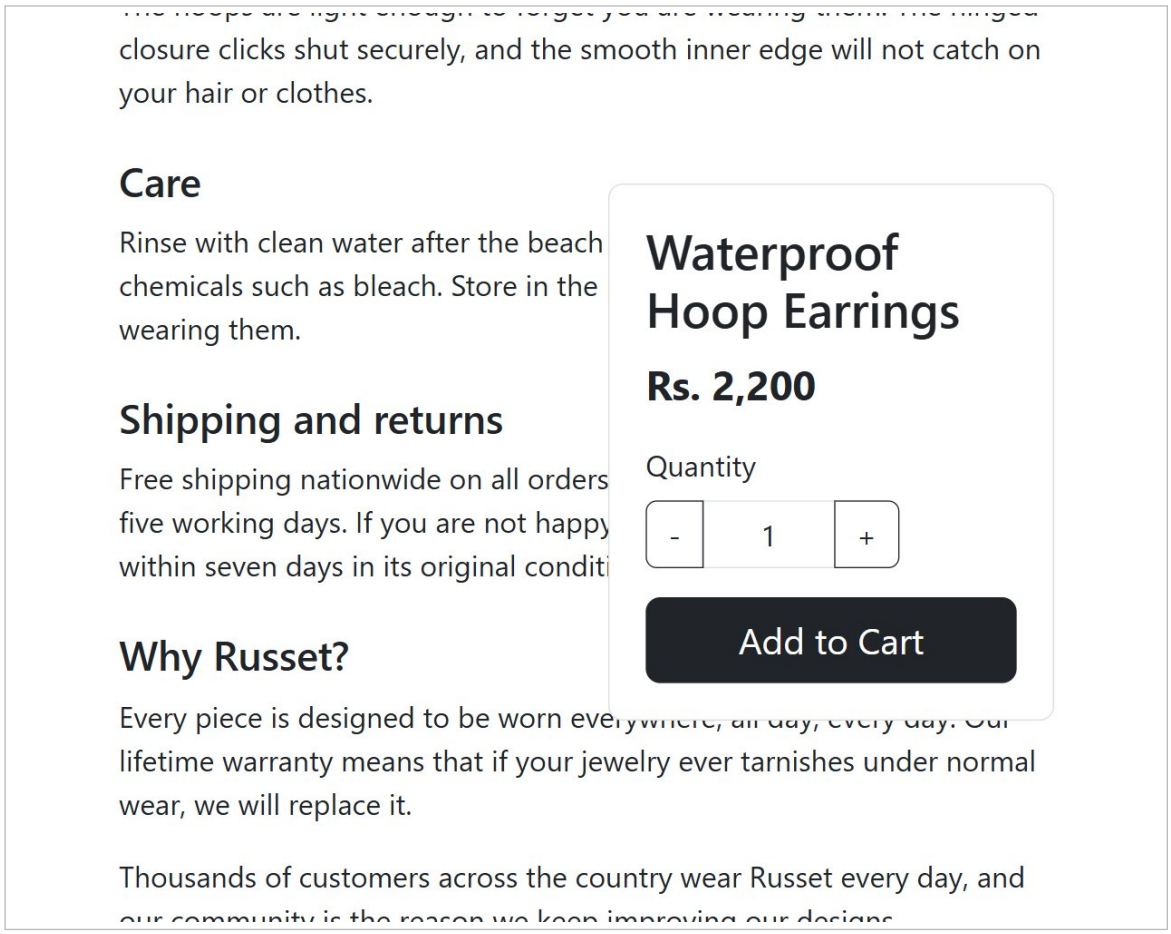


Task 1 - 1280px: Rings & Sets selected, remaining cards move up with no gaps

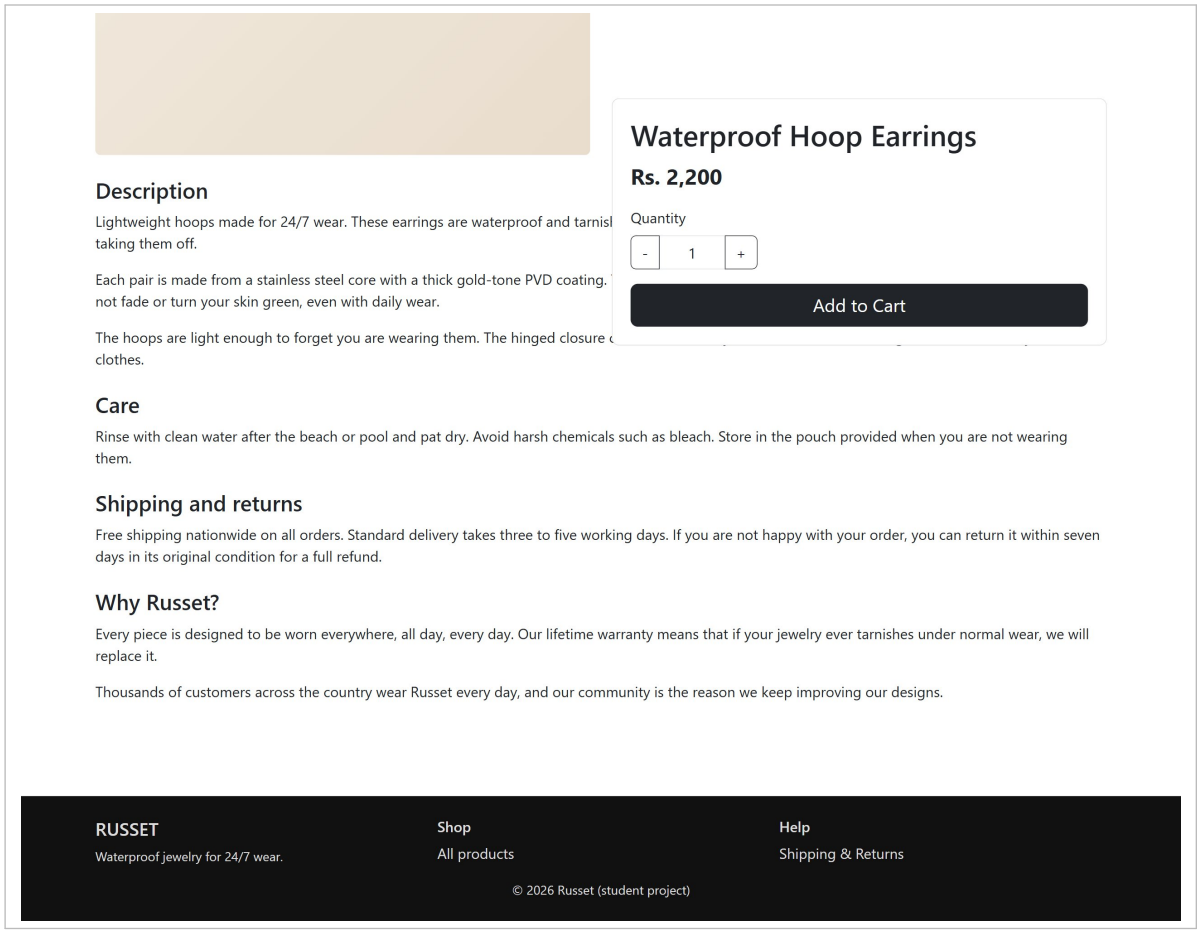
Task 2 - Product page with sticky details box



Task 2 - 375px: one column (details box, then description; image above)

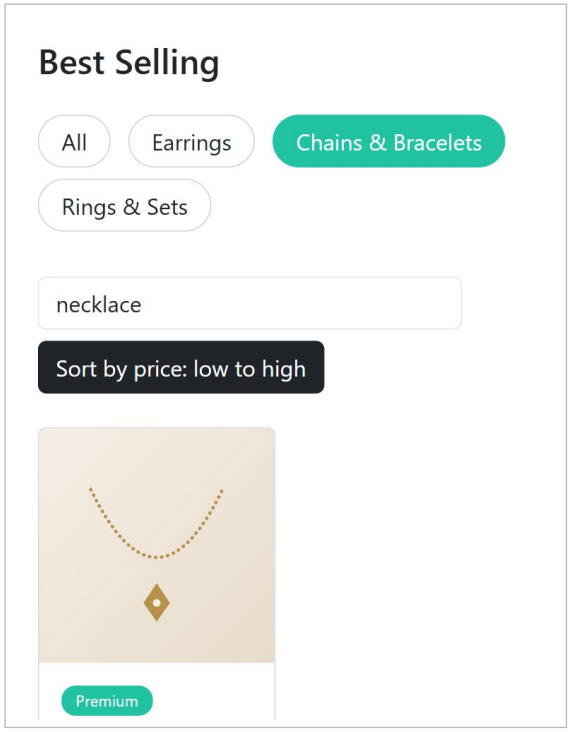


Task 2 - 624px (BP): two columns, details box sticky while scrolling

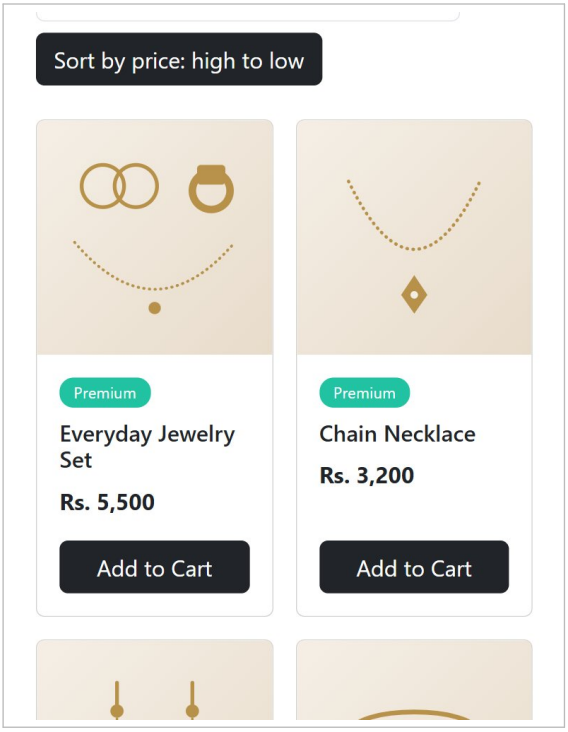


Task 2 - 1280px: scrolled down, details box stays 94px from the top

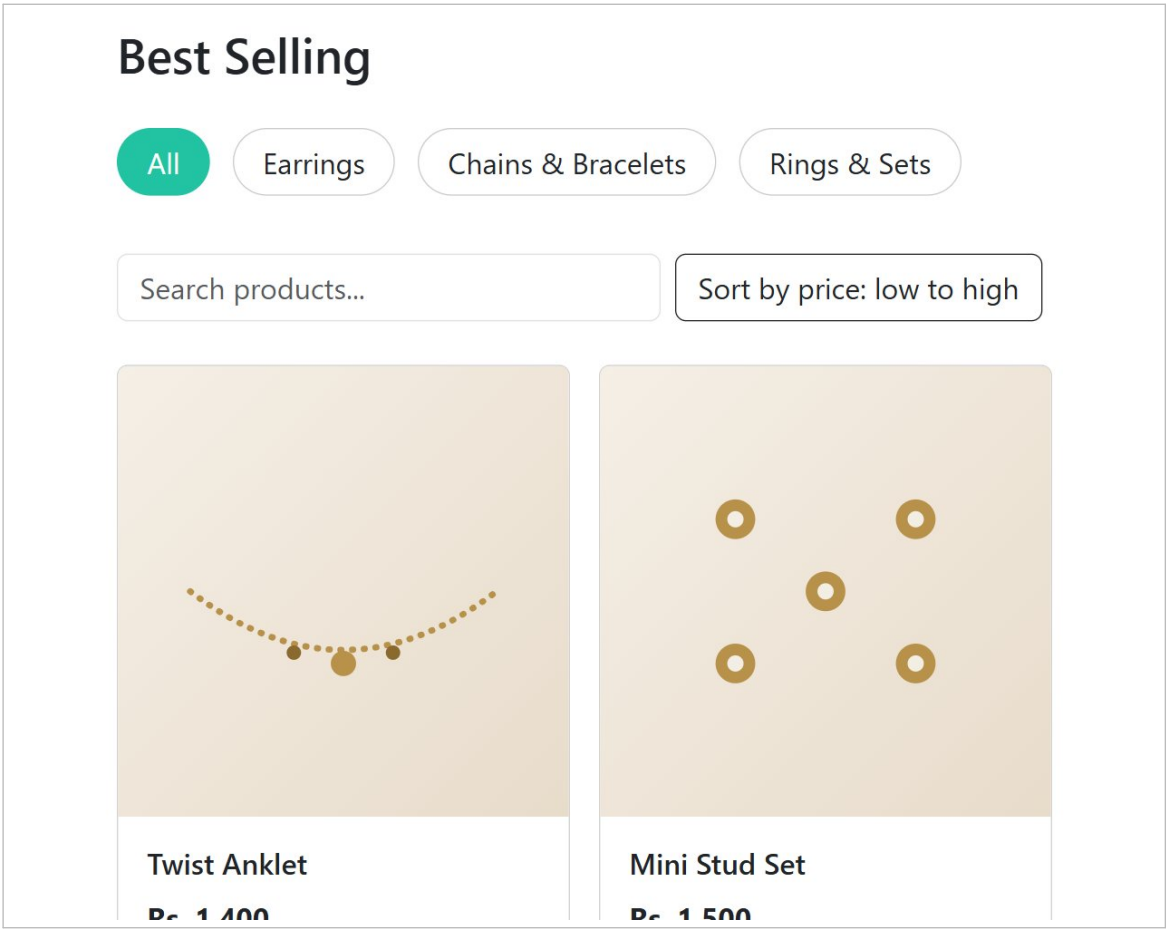
Task 3 - JavaScript search and price sort



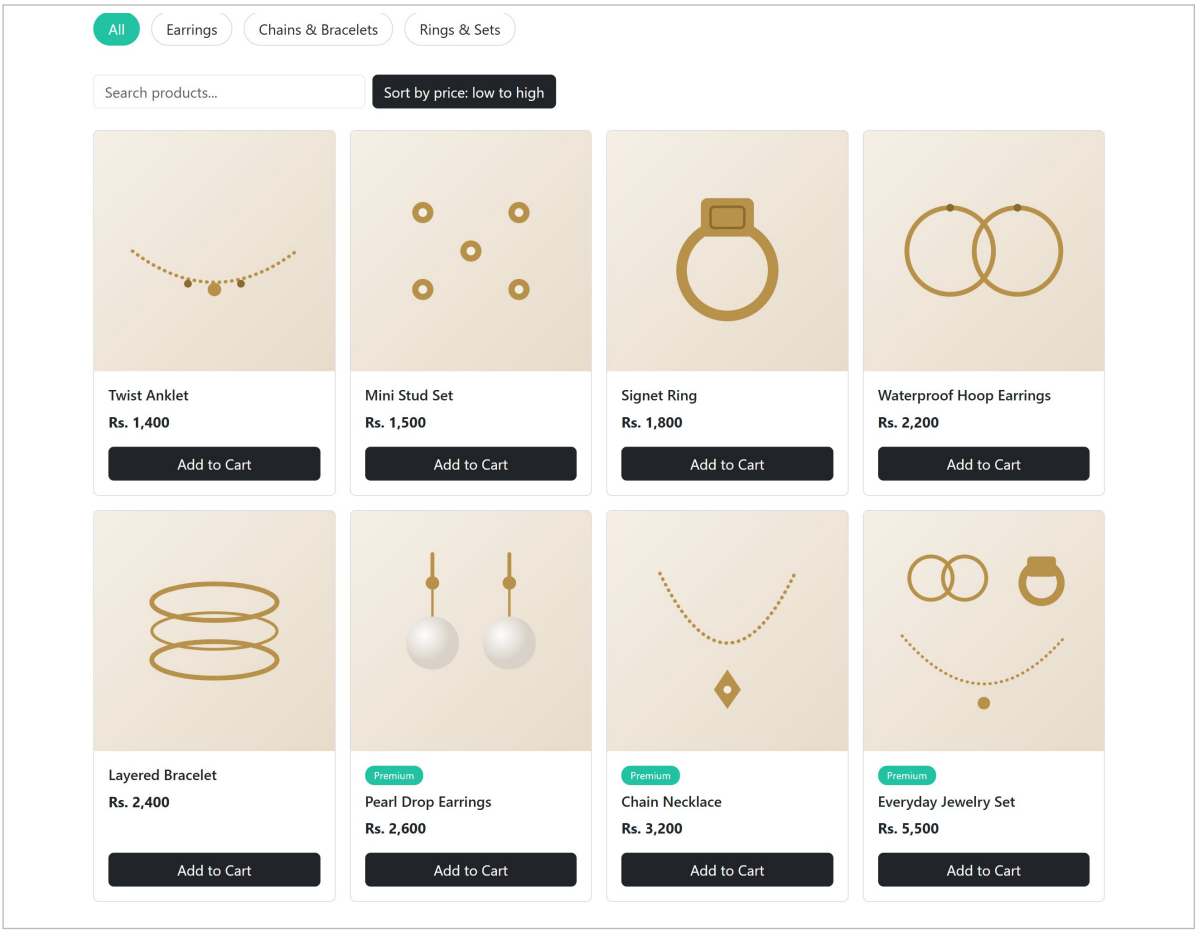
Task 3 - 375px: search 'necklace' together with the Chains & Bracelets filter



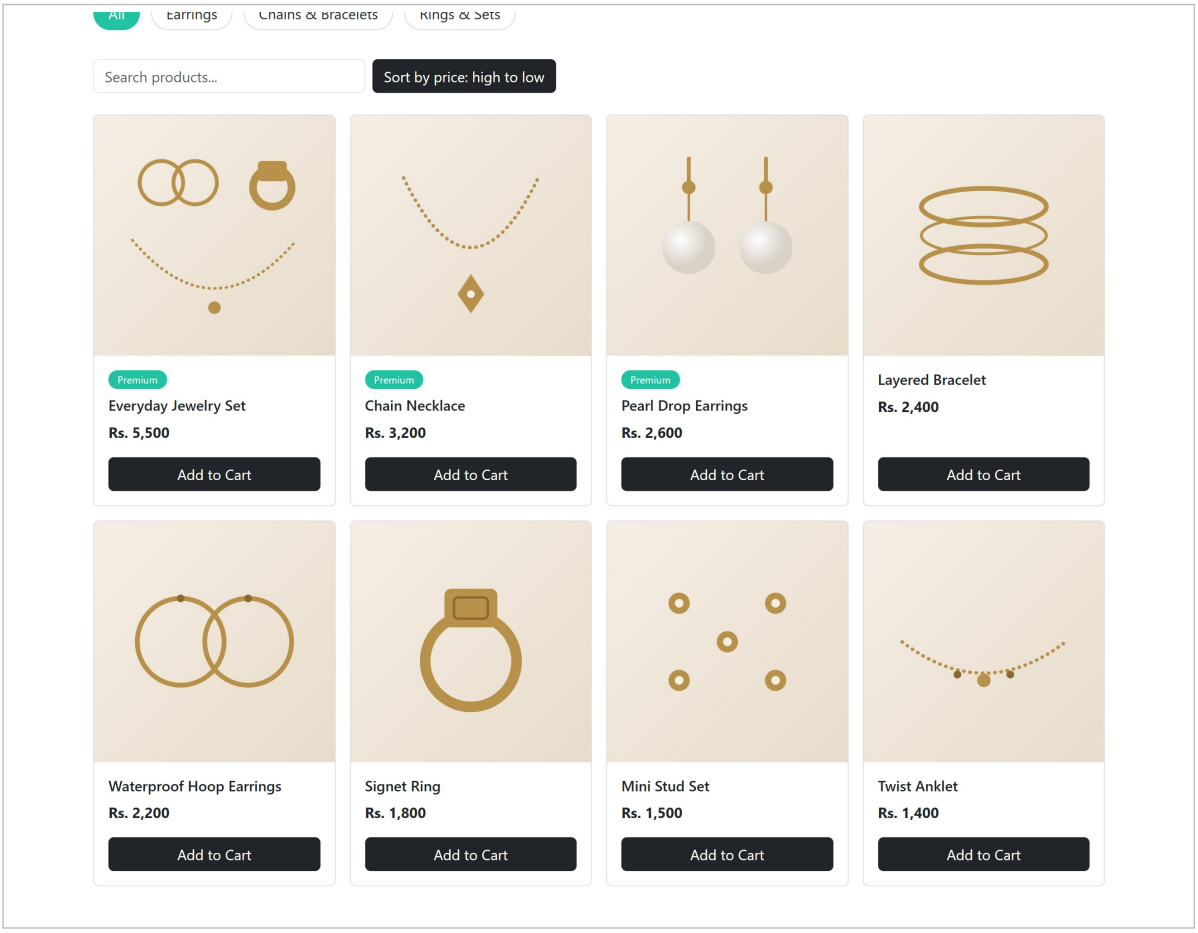
Task 3 - 375px: after clicking Sort by price (high to low), Premium labels visible



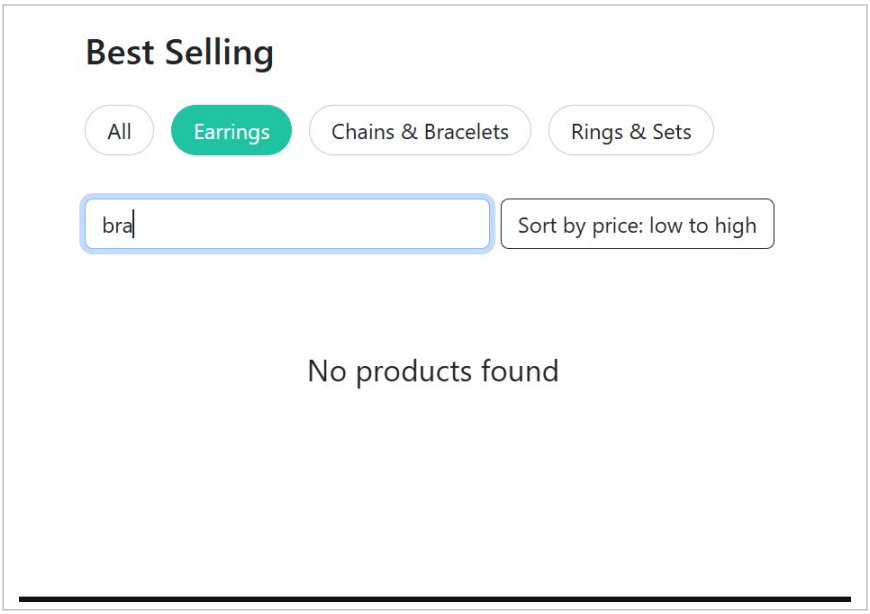
Task 3 - 624px (BP): after sorting low to high



Task 3 - 1280px: sorted low to high, 3 Premium labels (prices above Rs. 2,400)



Task 3 - 1280px: sorted high to low



Task 3 - 'No products found' message (Earrings filter + search 'bra')

Part 2 - Code added or changed

Task 1 - CSS-only category filter

index.html (filter markup)

```
<!-- TASK 1: hidden radio buttons remember which filter is selected.
     Same name="category" = only one selected at a time. "checked" = All on page load. -->
<input type="radio" name="category" id="cat-all" class="filter-radio" checked>
<input type="radio" name="category" id="cat-earrings" class="filter-radio">
<input type="radio" name="category" id="cat-chains" class="filter-radio">
<input type="radio" name="category" id="cat-rings" class="filter-radio">

<!-- TASK 1: visible buttons. Clicking a label selects the radio with the matching id. -->
<div class="filter-bar mb-4">
  <label for="cat-all">All</label>
  <label for="cat-earrings">Earrings</label>
  <label for="cat-chains">Chains &amp; Bracelets</label>
  <label for="cat-rings">Rings &amp; Sets</label>
</div>

<!-- Each product card carries a category class (earrings / chains / rings). Example: -->
<div class="col-6 col-md-4 col-lg-3 product earrings">
  <div class="card h-100">
    <a href="product.html"><img src="images/hoops.svg" class="card-img-top" alt="Waterproof Hoop Earrings"></a>
    <div class="card-body d-flex flex-column">
      <h3 class="card-title fs-6">Waterproof Hoop Earrings</h3>
      <p class="fw-bold">Rs. 2,200</p>
      <button class="btn btn-dark mt-auto">Add to Cart</button>
    </div>
  </div>
</div>
```

style.css

```
/* ===== TASK 1: CSS-only category filter ===== */

/* My accent colour: hue H = 168 */
:root {
  --accent: hsl(168, 70%, 45%);
}

/* Hide the radio circles but keep them in the page (they still work) */
.filter-radio {
  position: absolute;
  opacity: 0;
  width: 0;
  height: 0;
}

/* The label buttons (not selected) */
.filter-bar label {
  display: inline-block;
  cursor: pointer;
  padding: 6px 16px;
  margin: 0 8px 8px 0;
  border: 1px solid #ccc;
  border-radius: 20px;
}

/* The SELECTED label gets the accent colour */
#cat-all:checked ~ .filter-bar label[for="cat-all"],
#cat-earrings:checked ~ .filter-bar label[for="cat-earrings"],
#cat-chains:checked ~ .filter-bar label[for="cat-chains"],
#cat-rings:checked ~ .filter-bar label[for="cat-rings"] {
  background-color: var(--accent);
  border-color: var(--accent);
  color: white;
}

/* THE FILTER: if a radio is checked, hide every product that is NOT in its category */
#cat-earrings:checked ~ #product-grid .product:not(.earrings) { display: none; }
#cat-chains:checked   ~ #product-grid .product:not(.chains)   { display: none; }
#cat-rings:checked    ~ #product-grid .product:not(.rings)    { display: none; }
```

Task 2 - Product page layout with CSS Grid and sticky box

product.html (main section)

```
<main class="container py-5">

  <!-- TASK 2: CSS Grid container. Its 3 children are placed using named grid areas (see style.css) -->
  <div class="product-layout">

    <div class="product-image">
      <img src="images/hoops.svg" class="rounded" alt="Waterproof Hoop Earrings">
```

```
    </div>

    <!-- This is the box that stays sticky on wide screens -->
    <div class="product-details">
      <div class="details-box">
      <h1 class="h2">Waterproof Hoop Earrings</h1>
      <p class="fs-4 fw-bold">Rs. 2,200</p>

      <label for="qty" class="form-label">Quantity</label>
      <div class="input-group qty-box mb-3">
        <button class="btn btn-outline-dark" type="button">-</button>
        <input type="text" id="qty" class="form-control text-center" value="1">
        <button class="btn btn-outline-dark" type="button">+</button>
      </div>

      <button class="btn btn-dark btn-lg w-100">Add to Cart</button>
      </div>
    </div>

    <div class="product-description">
      <h2 class="h4">Description</h2>
      <p>Lightweight hoops made for 24/7 wear. These earrings are waterproof and tarnish-resistant, so you can
        swim, shower, work out and sleep in them without taking them off.</p>
      <p>Each pair is made from a stainless steel core with a thick gold-tone PVD coating. This coating is much
        harder than regular plating, which is why the color does not fade or turn your skin green, even with
        daily wear.</p>
      <p>The hoops are light enough to forget you are wearing them. The hinged closure clicks shut securely, and
         the smooth inner edge will not catch on your hair or clothes.</p>
      <h2 class="h4 mt-4">Care</h2>
      <p>Rinse with clean water after the beach or pool and pat dry. Avoid harsh chemicals such as bleach. Store
         in the pouch provided when you are not wearing them.</p>
      <h2 class="h4 mt-4">Shipping and returns</h2>
      <p>Free shipping nationwide on all orders. Standard delivery takes three to five working days. If you are
        not happy with your order, you can return it within seven days in its original condition for a full
        refund.</p>
      <h2 class="h4 mt-4">Why Russet?</h2>
      <p>Every piece is designed to be worn everywhere, all day, every day. Our lifetime warranty means that if
        your jewelry ever tarnishes under normal wear, we will replace it.</p>
      <p>Thousands of customers across the country wear Russet every day, and our community is the reason we
        keep improving our designs.</p>
    </div>

  </div>
</main>
```

style.css

```
/* ===== TASK 2: Product page with CSS Grid + named areas ===== */

/* BELOW 624px: one column, stacked in this order: image, details, description */
.product-layout {
  display: grid;
  grid-template-columns: 1fr;
  grid-template-areas:
    "image"
    "details"
    "description";
  gap: 24px;
}

/* Each child is assigned to a named area */
.product-image       { grid-area: image; }
.product-details     { grid-area: details; }
.product-description { grid-area: description; }

.product-image img {
  width: 100%;
  aspect-ratio: 4 / 5;      /* tall image, so the sticky box has room to travel */
  object-fit: cover;
}

/* The visible white box with the name, price, quantity and button */
.details-box {
  background: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 20px;
}

/* FROM 624px (BP = 600 + 24) AND WIDER */
@media (min-width: 624px) {
  .product-layout {
    grid-template-columns: 1fr 1fr;        /* two equal columns */
    grid-template-areas:
      "image details"                      /* top row: side by side */
      "description description";           /* bottom row: full width */
    align-items: start;
  }

  /* The wrapper stretches to the full height of the image row.
```

```
     The box can only stick inside it, so it never reaches the description. */
  .product-details {
    align-self: stretch;
  }

  .details-box {
    position: sticky;
    top: 94px;                              /* (N mod 40) + 70 = 24 + 70 = 94px */
  }
}
```

Task 3 - JavaScript search, sort and Premium label

index.html additions

```
<!-- search box and sort button (placed above the grid) -->
<div class="toolbar d-flex flex-wrap gap-2 mb-4">
  <input type="text" id="search" class="form-control" placeholder="Search products...">
  <button id="sort-btn" class="btn btn-outline-dark" type="button">Sort by price</button>
</div>

<!-- the grid gets an id so JavaScript can find it -->
<div class="row g-3" id="product-grid"> ... existing product cards ... </div>

<!-- message shown by JavaScript when nothing matches (d-none = hidden by default) -->
<p id="no-results" class="text-center fs-4 py-5 d-none">No products found</p>

<!-- at the bottom of the body, after the Bootstrap script -->
<script src="script.js"></script>
```

script.js

```
// ===== TASK 3: live search, price sort, Premium label =====
// I use the product cards that are already in the HTML. I do not create new ones.

const PREMIUM_LIMIT = 2400;   // N x 100, where N = 24

// Grab the elements I need from the page
const searchBox = document.querySelector('#search');
const sortBtn   = document.querySelector('#sort-btn');
const grid      = document.querySelector('#product-grid');
const noResults = document.querySelector('#no-results');
const cards     = document.querySelectorAll('.product');            // all 8 cards
const radios    = document.querySelectorAll('input[name="category"]'); // 4 filter radios

// Read the product name from a card, in lower case
function getName(card) {
  return card.querySelector('.card-title').textContent.toLowerCase();
}

// Read the price from a card: "Rs. 2,200" -> remove everything except digits -> 2200
function getPrice(card) {
  const text = card.querySelector('.card-body p').textContent;
  return Number(text.replace(/[^0-9]/g, ''));
}

// ----- 1. SEARCH (works together with the CSS category filter) -----
function updateVisibility() {
  const text = searchBox.value.trim().toLowerCase();
  let visibleCount = 0;

  cards.forEach(card => {
    const matchesSearch = getName(card).includes(text);

    // Add/remove a class that hides the card (CSS: .hidden-by-search { display: none; })
    card.classList.toggle('hidden-by-search', !matchesSearch);

    // A card is really visible only if NEITHER the search NOR the category filter hides it.
    // getComputedStyle tells me the final display value after all CSS is applied.
    if (getComputedStyle(card).display !== 'none') {
      visibleCount++;
    }
  });

  // No visible cards -> hide the grid and show the message (and the reverse)
  grid.classList.toggle('d-none', visibleCount === 0);
  noResults.classList.toggle('d-none', visibleCount !== 0);
}

// Run on every keystroke
searchBox.addEventListener('input', updateVisibility);

// Also run when the category changes, because the category can also leave zero matches
radios.forEach(radio => radio.addEventListener('change', updateVisibility));

// ----- 2. SORT BY PRICE (click 1: low to high, click 2: high to low, ...) -----
let ascending = true;

sortBtn.addEventListener('click', () => {
```

```
  // Turn the NodeList into an array so I can use .sort()
  const sorted = Array.from(cards).sort((a, b) =>
    ascending ? getPrice(a) - getPrice(b) : getPrice(b) - getPrice(a)
  );

  // appendChild MOVES an existing element, so adding them in sorted order re-orders the grid
  sorted.forEach(card => grid.appendChild(card));

  sortBtn.textContent = ascending ? 'Sort by price: low to high' : 'Sort by price: high to low';
  ascending = !ascending;   // flip direction for the next click
});

// ----- 3. PREMIUM LABEL (added when the page loads) -----
cards.forEach(card => {
  if (getPrice(card) > PREMIUM_LIMIT) {
    const label = document.createElement('span');   // make a new <span>
    label.className = 'premium-label';
    label.textContent = 'Premium';
    card.querySelector('.card-body').prepend(label); // put it at the top of the card body
  }
});
```

style.css

```
/* ===== TASK 3: search, sort, premium label ===== */

/* Hides a card when the search doesn't match (JS adds/removes this class) */
.hidden-by-search {
  display: none;
}

/* Keep the search box a sensible width */
.toolbar input {
  max-width: 300px;
}

/* Small "Premium" label added by script.js */
.premium-label {
  align-self: flex-start;
  background-color: var(--accent);
  color: white;
  font-size: 0.7rem;
  padding: 2px 10px;
  border-radius: 10px;
  margin-bottom: 8px;
}
```